

Project Overview

✦ Project Title:

Student Information System

✦ Project Type:

Console-Based Java Application

✦ Project Description:

The Student Information System is a Java-based console application designed to manage student records efficiently. The system allows users to perform operations such as adding, viewing, updating, deleting, and searching student records.

The application follows Object-Oriented Programming principles and uses dynamic data storage through `ArrayList`.

🎯 Project Goals & Objectives

- Develop a structured student record management system.
 - Implement CRUD operations (Create, Read, Update, Delete).
 - Validate user inputs (age and grade).
 - Provide formatted and readable output.
 - Implement search functionality.
 - Apply exception handling to prevent crashes.
 - Demonstrate use of OOP and Java Collections.
-

Setup Instructions (Installation Guide)

◆ System Requirements

Hardware:

- 4GB RAM minimum
- Any modern PC

Software:

- Java JDK 8 or above
 - VS Code / IntelliJ / Eclipse
 - Command Prompt / Terminal
-

◆ Step 1: Install Java

Download and install JDK.

Verify installation:

```
java -version  
javac -version
```

◆ Step 2: Create Project File

1. Open IDE.
2. Create file:

StudentManagementSystem.java

3. Paste full project code.
 4. Save file.
-

◆ Step 3: Compile Program

```
javac StudentManagementSystem.java
```

◆ Step 4: Run Program

```
java StudentManagementSystem
```

Program starts with menu interface.

Code Structure & Architecture

File Structure

StudentManagementSystem.java

```
└─ class Student
└─ class StudentManagementSystem (Main Class)
```

□ Architecture Overview

The project follows a simple two-class architecture:

1 Student Class

- Stores student attributes.
- Performs data validation.
- Displays formatted output.

2 StudentManagementSystem Class

- Contains main method.
 - Stores students in ArrayList.
 - Handles user interaction.
 - Performs CRUD operations.
-

★ Data Structure Used

✓ ArrayList<Student>

Reason:

- Dynamic size
 - Easy insertion & deletion
 - No fixed memory limit
 - Built-in methods for searching and iteration
-

★ Algorithms Used

◆ Add Student

1. Take input
2. Validate data
3. Create object
4. Add to ArrayList

Time Complexity: $O(1)$

◆ View Students

1. Iterate through ArrayList
2. Print formatted output

Time Complexity: $O(n)$

◆ Update Student

1. Search by ID
2. Modify attributes

Time Complexity: $O(n)$

◆ Delete Student

1. Use Iterator
2. Match ID
3. Remove safely

Time Complexity: $O(n)$

◆ Search Student

1. Choose ID or Name
2. Loop through list
3. Compare values

Time Complexity: $O(n)$

Technical Details

◆ Concepts Used

- Object-Oriented Programming (Encapsulation)
- Constructors
- Getters & Setters

- ArrayList
 - Exception Handling
 - Iterator
 - printf Formatting
 - Switch-case control structure
-

◆ Validation Logic

Age Validation:

```
if (age > 0)
```

Grade Validation:

```
if (grade >= 0 && grade <= 100)
```

◆ Error Handling

Used:

```
try-catch (InputMismatchException)
```

Prevents crash when user enters invalid data type.

User Manual

◆ Starting the Program

Run program → Menu appears:

1. Add Student
 2. View Students
 3. Update Student
 4. Delete Student
 5. Search Student
 6. Exit
-

◆ Functional Guide

➤ Add Student

- Enter student details.
- System validates age & grade.

➤ View Students

- Displays formatted table.

➤ Update Student

- Enter Student ID.
- Modify details.

➤ Delete Student

- Enter Student ID.
- Record removed.

➤ Search Student

- Search by ID or Name.
-

Visual Documentation (Screenshots)

Main Menu

```
===== STUDENT INFORMATION SYSTEM =====  
1. Add Student  
2. View All Students  
3. Update Student  
4. Delete Student  
5. Search Student  
6. Exit
```

Add Student,View Students,Update Student info,Search Student

```
===== STUDENT INFORMATION SYSTEM =====
1. Add Student
2. View All Students
3. Update Student
4. Delete Student
5. Search Student
6. Exit
Enter choice: 1
Enter ID: 423
Enter Name: ajay
Enter Age: 20
Enter Grade: 9.81
Enter Contact: ajk@gmail.com
Student added successfully!

===== STUDENT INFORMATION SYSTEM =====
1. Add Student
2. View All Students
3. Update Student
4. Delete Student
5. Search Student
6. Exit
Enter choice: 1
Enter ID: 421
Enter Name: Arun
Enter Age: 19
Enter Grade: 9.5
Enter Contact: abj@gmail.com
Student added successfully!

===== STUDENT INFORMATION SYSTEM =====
1. Add Student
2. View All Students
3. Update Student
4. Delete Student
5. Search Student
6. Exit
Enter choice: 2
ID          Name          Age    Grade    Contact
-----
423         ajay          20     9.81     ajk@gmail.com
421         Arun           19     9.50     abj@gmail.com

===== STUDENT INFORMATION SYSTEM =====
1. Add Student
2. View All Students
3. Update Student
4. Delete Student
5. Search Student
6. Exit
Enter choice: 3
Enter Student ID to update: 423
Enter New Name: Akash
Enter New Age: 20
Enter New Grade: 9.21
Enter New Contact: ak@gmail.com
Student updated successfully!
```

Invalid Input Handling

```
===== STUDENT INFORMATION SYSTEM =====
1. Add Student
2. View All Students
3. Update Student
4. Delete Student
5. Search Student
6. Exit
Enter choice: 1
Enter ID: abbh
Enter Name: sandy
Enter Age: 21
Enter Grade: 150
Enter Contact: 21
Invalid grade,Set default grade = 0
Student added successfully!
```

```
===== STUDENT INFORMATION SYSTEM =====
1. Add Student
2. View All Students
3. Update Student
4. Delete Student
5. Search Student
6. Exit
Enter choice: 7
Invalid choice!
```